



Facultad de Ingeniería

MODELADO, IMPLEMENTACIÓN Y ANÁLISIS DE PROBLEMAS NP-COMPLETOS

Problema 13: Clique Cover

Autores:

Matias Alonso Santos Soza

Jose Amaro Romero Moreno

David Alejandro Cortez Leal

Profesor:

Luis Fuentes Larenas

Santiago, Chile

2026

Índice

1. Introducción	3
2. Definición del problema	4
2.1. Descripción en lenguaje natural	4
2.2. Definición formal	4
2.3. Ejemplo ilustrativo	4
2.4. Aplicaciones	5
3. Modelado matemático	6
3.1. Conjuntos y parámetros	6
3.2. Variables de Decisión	6
3.3. Restricciones	6
3.4. Función objetivo	7
3.5. Versión de decisión	7
4. Complejidad y contexto	8
4.1. Pertenencia a NP	8
4.2. NP -completitud	8
4.3. Relación con otros problemas	9
5. Implementación	10
5.1. Enfoque base: Backtracking	10
5.2. Enfoque mejorado: Greedy	10
5.3. Implementación en C++	11
5.4. Implementación en Java	11
5.5. Implementación en Python	11
6. Análisis experimental	12
6.1. Solución base Backtracking	12
6.1.1. Resultados	12
6.1.2. Escalabilidad	13
6.1.3. Análisis	13
6.2. Solución mejorada heurísticas Greedy	13
6.2.1. Resultados	13
6.2.2. Escalabilidad	14
6.2.3. Análisis	15
6.3. Comparación de Enfoques	15
6.3.1. Análisis de Calidad de la Solución (Diferencia de Cliques)	15
6.3.2. Discusión del Compromiso Tiempo vs. Calidad	16

1. Introducción

En el presente documento se verá el problema computacional de clase NP-completo llamado Clique Cover, para ello se presentará el problema en sí con su explicación, seguido de su modelo matemático, luego se explicará su razón de ser de la clase NP-completo con su NP-completitud y su relación con otros problemas.

Posterior a eso se experimentará con este problema mediante dos soluciones, una base y otra mejorada, y cada una de estas ejecutadas en 3 lenguajes de programación, los cuales serán Java, Python y C++.

Una vez obtenidos los resultados se analizarán sus tiempos de ejecución y comportamiento, esto para ver las diferencias de enfoque entre cada lenguaje con cada solución para relacionarlos a la complejidad del problema y determinar las ventajas y complicaciones de cada lenguaje y solución.

2. Definición del problema

En esta sección se procederá a presentar una descripción general del problema *Clique Cover* empezando con una explicación intuitiva seguido de su formulación formal, un ejemplo ilustrativo y algunas de sus aplicaciones mas relevantes.

2.1. Descripción en lenguaje natural

El problema de *Clique Cover* tiene como propósito dividir los vértices de los grafos en tantos subconjuntos como sea posible, de tal manera de que cada subconjunto sea un clique. Es decir, todos los vértices del mismo están relacionados entre sí.

De manera intuitiva, el objetivo seria entonces dividir los vértices en subconjuntos totalmente compatibles, logrando además minimizar la cantidad total de subconjuntos.

2.2. Definición formal

Sea $G = (V, E)$ un grafo no dirigido.

Un subconjunto $C \subseteq V$ es una clique si:

$$\forall u, v \in C, u \neq v \Rightarrow \{u, v\} \in E$$

Entrada: Un grafo no dirigido $G = (V, E)$.

Salida: Encontrar el mínimo número k tal que existe una partición de V en subconjuntos $C_1, C_2, \dots, C_k$, donde cada C_i es una clique.

Esto es, encontrar k tal que:

$$V = C_1 \cup C_2 \cup \dots \cup C_k$$

$$C_i \cap C_j = \emptyset \quad \text{para todo } i \neq j$$

y cada C_i es una clique.

Versión de decisión: Dado un entero k , determinar si existe una partición de V en a lo más k cliques.

2.3. Ejemplo ilustrativo

Considérese el grafo $G = (V, E)$ con:

$$V = \{1, 2, 3, 4\}$$

$$E = \{(1, 2), (2, 3), (1, 3), (3, 4)\}$$

El subconjunto $\{1, 2, 3\}$ forma una clique, ya que todos sus vértices están conectados entre sí. El vértice 4 solo está conectado con el vértice 3, por lo que no puede pertenecer a esa misma clique.

Una posible cobertura por cliques es:

$$\{1, 2, 3\}, \quad \{4\}$$

Por lo tanto, el número mínimo de cliques necesarias para cubrir el grafo es $k = 2$.

2.4. Aplicaciones

El problema de *Clique Cover* se puede encontrar en muchos ámbitos prácticos en los cuales se pretenden agrupar elementos compatibles entre sí. Algunas aplicaciones destacadas son:

- **Análisis de redes sociales:** detección de comunidades en la que los individuos se encuentra todos conectados entre ellos.
- **Asignación de recursos:** agrupación de las tareas o de las entidades que pueden ser ejecutadas de forma concurrente.
- **Bioinformática:** detección de grupos de genes o proteínas que presentan interacciones completas.
- **Optimización y teoría de grafos:** formulaciones equivalentes como la coloración del grafo complemento, las cuales se utilizan en problemas de planificación o en particionamiento.

3. Modelado matemático

El problema abordado puede formularse como un modelo de programación lineal entera. El objetivo de este problema consiste en asignar un vértice a un clique, de forma que se cumplan las condiciones de compatibilidad y se minimice el número total de cliques utilizadas.

3.1. Conjuntos y parámetros

En esta sección se define los distintos elementos a considerar como básicos en el modelo, el grafo entrada y los índices a gestionar, los cuales se utilizan para representar las posibles cliques.

Sea $G = (V, E)$ un grafo no dirigido, donde:

- V es el conjunto de vértices, con $|V| = n$
- E es el conjunto de aristas
- $K = \{1, 2, \dots, n\}$ es el conjunto de posibles cliques (cota superior)

3.2. Variables de Decisión

Las variables de decisión permiten representar la asignación de los vértices a las cliques y determinar cuáles de ellos se encuentran efectivamente en la solución.

- $x_{ik} \in \{0, 1\}$: vale 1 si el vértice $i \in V$ es asignado a la clique $k \in K$, y 0 en caso contrario.
- $y_k \in \{0, 1\}$: vale 1 si la clique $k \in K$ es utilizada, y 0 en caso contrario.

3.3. Restricciones

Las siguientes restricciones garantizan que la solución represente una partición válida en cliques, la cual respeta la asignación de los vértices que dan a la solución y la forma del grafo.

1. Asignación única de vértices

Cada vértice debe pertenecer exactamente a una clique:

$$\sum_{k \in K} x_{ik} = 1 \quad \forall i \in V$$

2. Activación de cliques

Un vértice solo puede ser asignado a una clique si esta está activa:

$$x_{ik} \leq y_k \quad \forall i \in V, \forall k \in K$$

3. Condición de clique

Dos vértices que no están conectados no pueden pertenecer a la misma clique:

$$x_{ik} + x_{jk} \leq 1 \quad \forall (i, j) \notin E, i \neq j, \forall k \in K$$

3.4. Función objetivo

Con las variables de decisión se tienen las condiciones de presentar la asignación de los vértices a las cliques y determinar si estas efectivamente aparecen en la solución.

Minimizar la cantidad de cliques necesarias para cubrir los vértices de un grafo:

$$\min \sum_{k \in K} y_k$$

3.5. Versión de decisión

La forma de decisión del problema ofrece la posibilidad de contrastar la existencia de una solución restringida a un número máximo de cliques

Dado un entero p , se desea determinar si existe una solución factible tal que:

$$\sum_{k \in K} y_k \leq p$$

cumpliendo todas las restricciones anteriores.

4. Complejidad y contexto

El problema en cuestión está contenido en la clase de problemas NP-completo. Se argumenta su pertenencia a la clase NP y su NP completitud.

4.1. Pertenencia a NP

Un problema pertenece a la clase NP si, dadas las soluciones, es posible verificar si estas soluciones son correctas en tiempo polinomial. El problema se puede ejemplificar suponiendo que se tiene como solución un conjunto de K subconjuntos, que son los que particionan el conjunto de los vértices V . Para verificar que una solución dada cumple con la propiedad de solución del problema, la solución debe cumplir las siguientes condiciones:

- Verificar que cada vértice pertenece a exactamente un subconjunto.
- Para cada subconjunto, comprobar que todos los pares de vértices están conectados entre sí, es decir, que cada subconjunto produce una clique.

La verificación de la primera condición se lleva a cabo en tiempo polinomial, porque consiste en comprobar si las aristas entre los pares de vértices dentro de cada subconjunto.

Por todo lo anterior se asume que el problema de *Clique Cover* pertenece a la clase NP .

4.2. NP -completitud

Para demostrar que el problema es NP -completo hay que comprobar que es NP -duro. Esto se hace llevando a cabo una reducción polinomial desde un problema que se sabe que es NP -completo.

En esta ocasión se considerará el problema de *Graph Coloring*. Este problema consiste en asignar colores a los vértices de un grafo de manera que no existan dos vértices adyacentes que estén coloreados del mismo color, y que además estén representados con el menor número de colores posibles.

La demostración se realizará considerando la relación existente entre un grafo G y su grafo complemento $\overline{G}$, el cual está definido sobre el mismo conjunto de vértices, donde existe una arista entre dos vértices si y solo si dicha arista no pertenece a E .

En este contexto, se observa que:

- Un conjunto de vértices forma un clique G si y solo si se forma un conjunto independiente en $\overline{G}$.
- Cubrir los vértices de G con K cliques es equivalente a colorear el grafo $\overline{G}$ con sus K colores.

Dado que el problema *Graph Coloring* es *NP-completo* y existe una transformación polinomial entre este y el problema de *Clique Cover*, se concluye que *Clique Cover* es *NP-duro*.

Como además se demostró anteriormente que dicho problema pertenece a *NP* es posible llegar a la conclusión de que *Clique Cover* es *NP-completo*.

4.3. Relación con otros problemas

El problema abordado tiene una relación estrecha con otros problemas de la teoría de grafos en estos se incluyen *Vertex Cover* y *Graph Coloring*. Estos problemas forman la parte fundamental del conjunto de problemas de *NP-completos* y comparten estructuras y transformaciones similares.

Cuadro 1: Relación entre Clique Cover, Vertex Cover y Graph Coloring

Problema	Definición	Objetivo	Relación con otros
Clique Cover	Particionar los vértices en cliques (subgrafos completos)	Minimizar el número de cliques	Equivalente a Graph Coloring en el grafo complemento
Vertex Cover	Conjunto de vértices que cubre todas las aristas	Minimizar el número de vértices	Complementario de Independent Set
Graph Coloring	Asignar colores a vértices sin que adyacentes compartan color	Minimizar el número de colores	Equivalente a Clique Cover en el complemento

Fuente: Elaboración propia basada en *Computers and Intractability: A Guide to the Theory of NP-Completeness* [1].

5. Implementación

En esta sección se plantea la descripción de la forma de implementar los enfoques propuestos para resolver el problema *Clique Cover* en los lenguajes de programación C++, Java y Python, teniendo tanto una solución base como una solución mejorada, gracias a las técnicas de backtracking a las estrategias heurísticas.

5.1. Enfoque base: Backtracking

La solución base en los tres lenguajes corresponde a un algoritmo de *backtracking*, el cual explora un espacio de soluciones de manera sistemática.

El algoritmo construye la solución a partir de la asignación de vértices a cliques de manera incrementable. En cada paso:

- Se selecciona un vértice no asignado.
- Se intenta ubicarlo en alguna de las cliques existentes.
- Se verifica que el vértice sea adyacente a todos los miembros de la clique.
- Si no es posible, se crea una nueva clique.

Si en algún momento no es posible continuar, el algoritmo hace retroceso *backtrack* y prueba otra configuración.

Este enfoque garantiza que se encuentre la solución óptima, pero también presenta una complejidad exponencial.

5.2. Enfoque mejorado: Greedy

Para lograr mejorar el rendimiento, se incorporó una estrategia de tipo *Greedy* que guía el proceso de la búsqueda.

En vez de explorar todas las configuraciones posibles, este algoritmo:

- Intenta primero asignar vértices a cliques ya existentes.
- Minimiza la creación de nuevas cliques.
- Prioriza configuraciones que reduzcan rápidamente el número total de cliques.

Ademas, en las implementaciones de Java y Python se utiliza una estrategia de *profundización iterativa*, donde se resuelve el problema de decisión:

¿Es posible cubrir el grafo con K cliques?

Comenzando primero desde valores pequeños de K . El algoritmo se detiene cuando encuentra el primer valor que sea factible, el cual corresponde a la solución óptima.

5.3. Implementación en C++

La implementación en C++ combina tanto backtracking como poda (*branch and bound*):

- Se mantiene una cota superior del número de cliques.
- Se descartan ramas que no pueden mejorar la mejor solución encontrada.
- Se utilizan estructuras eficientes como `bitset` para verificar adyacencias.

Este ultimo enfoque es mas exhaustivo, pero altamente óptimo a nivel de ejecución.

5.4. Implementación en Java

La implementación en Java utiliza la técnica de backtracking guiado por una profundización iterativa:

- Se evalúan valores crecientes de K .
- Se detiene al encontrar la primera solución factible.
- Se beneficia de optimizaciones de la JVM (JIT).

Este ultimo enfoque reduce de manera significativa en espacio de la búsqueda.

5.5. Implementación en Python

La implementación en Python sigue exactamente la misma lógica de Java:

- Backtracking con criterio de parada temprana.
- Exploración limitada del espacio de soluciones.

Sin embargo, esta implementación debido a su naturaleza interpretada presenta un menor rendimiento en comparación a los otros lenguaje de programación.

6. Análisis experimental

Se analizaron los resultados que se obtuvieron al ejecutar las implementaciones en C++, Java y Python sobre 53 instancias, comparando los tiempos de ejecución y el comportamiento práctico.

6.1. Solución base Backtracking

En primera instancia se analizará los resultados obtenidos a través de la metodología backtracking con el fin de determinar las diferencias entre los enfoques mencionados con anterioridad.

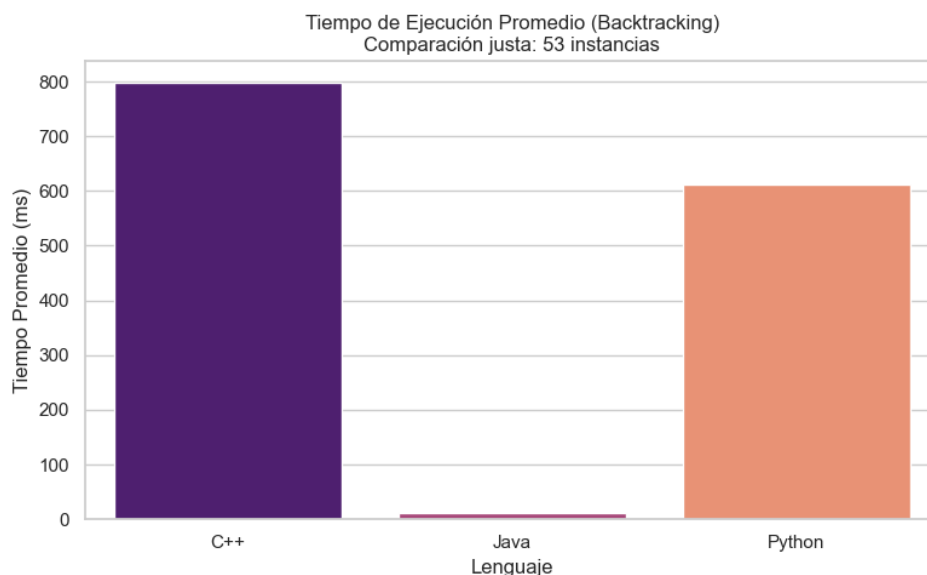
6.1.1. Resultados

Los resultados obtenidos tras la ejecución de estas tres implementaciones se detallan a continuación mediante una tabla y gráfico comparativo.

Lenguaje	Tiempo Promedio (ms)	Tipo de Enfoque	Observación
Java	~10.2	Terminación temprana	Más rápido
Python	~610.5	Terminación temprana	Intermedio
C++	~800.2	Exhaustivo con poda	Más lento

Fuente: Elaboración propia basada en los resultados obtenidos tras ejecutar tools benchmark.

Figura 1: Tiempo de ejecución promedio en C++, Java y Python (53 instancias).



Fuente: Elaboración propia basada en los resultados obtenidos tras ejecutar tools benchmark.

Como es posible observar en la Figura 1, Java presenta un menor tiempo de ejecución, seguido por Python y finalmente C++

6.1.2. Escalabilidad

El problema presenta complejidad exponencial por lo que todos los enfoques aumentan su tiempo de ejecución cuando crece el tamaño del grafo. Sin embargo, la terminación temprana permite a Java y Python escalar mejor en la practica.

6.1.3. Análisis

La principal diferencia radica en el enfoque de los algoritmos. Java y Python utilizan una estrategia de decision con terminación temprana, deteniéndose al momento de encontrar la solución óptima. Por otro lado C++ realiza una exploración mas exhaustiva, lo cual incrementa su tiempo de ejecución.

Entre Java y Python, la ventaja de la implementación de Java se debe a la optimización JIT y mejor manejo de la memoria.

6.2. Solución mejorada heurísticas Greedy

Ahora se procederá a analizar la solución mejorada que en este caso sera mediante la técnica Greedy (heurísticas avara)

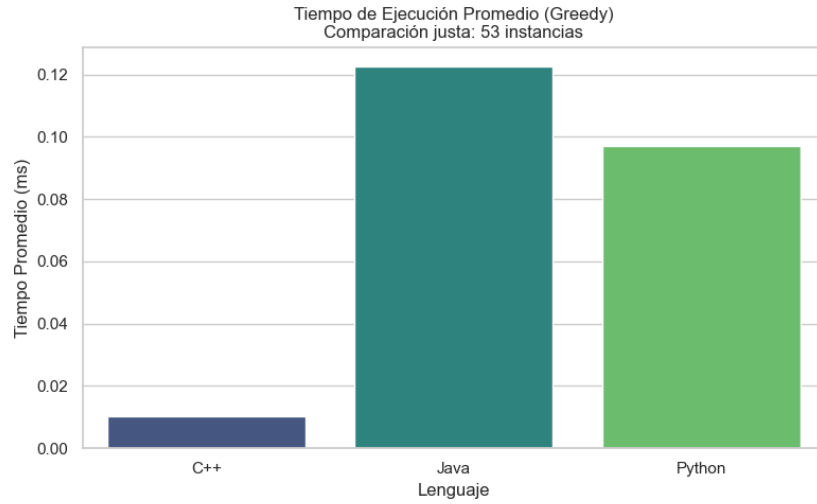
6.2.1. Resultados

Los resultados obtenidos tras la ejecución de estas tres implementaciones se detallan a continuación mediante una tabla y gráfico comparativo.

Lenguaje	Tiempo Promedio (ms)	Tipo de Enfoque	Observación
C++	~0.01	Greedy directo	Más rápido
Python	~0.09	Greedy con ordenamiento	Intermedio
Java	~0.12	Greedy secuencial	Más lento

Fuente: Elaboración propia basada en los resultados obtenidos tras ejecutar tools benchmark.

Figura 2: Tiempo de ejecución promedio del algoritmo Greedy (53 instancias).



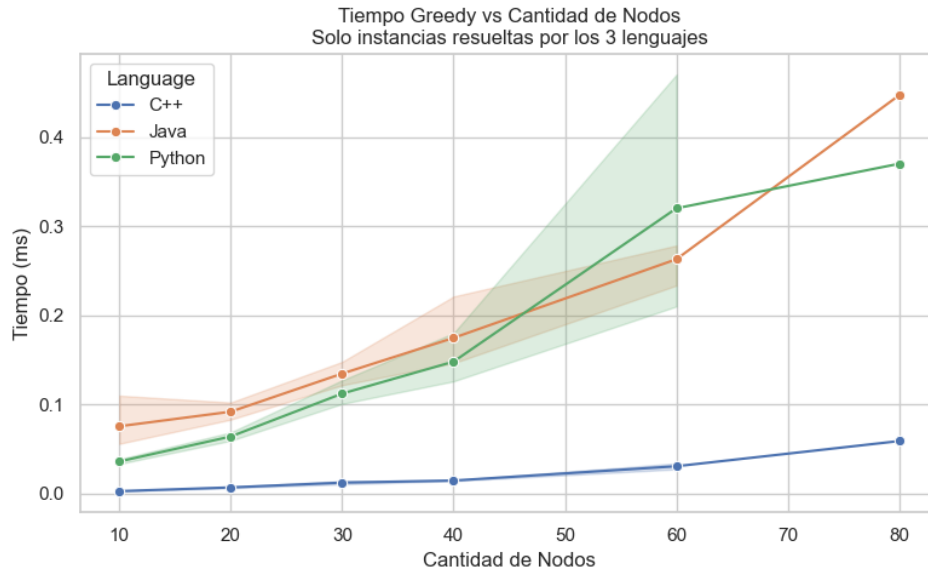
Fuente: Elaboración propia basada en los resultados obtenidos tras ejecutar tools benchmark.

Como se observa en la Figura 2, C++ presenta el menor tiempo, seguido por Python y Java.

6.2.2. Escalabilidad

En esta sección se analizará en como varía el tiempo de ejecución en función del tamaño del grafo.

Figura 3: Tiempo de ejecución en función de la cantidad de nodos.



Fuente: Elaboración propia basada en los resultados obtenidos tras ejecutar tools benchmark.

La Figura 3 muestra un crecimiento aproximadamente lineal del tiempo de ejecución en los tres lenguajes, con diferencias en la pendiente.

6.2.3. Análisis

Las diferencias se explican principalmente por la implementación:

- C++ logra el mejor rendimiento gracias a su ejecución nativa y bajo overhead.
- Python mejora su desempeño mediante el ordenamiento previo, reduciendo conflictos.
- Java presenta mayor tiempo debido a la sobrecarga de la JVM en ejecuciones cortas.

A diferencia del backtracking, el enfoque Greedy no garantiza optimalidad, pero reduce significativamente el tiempo de ejecución al evitar la exploración exhaustiva.

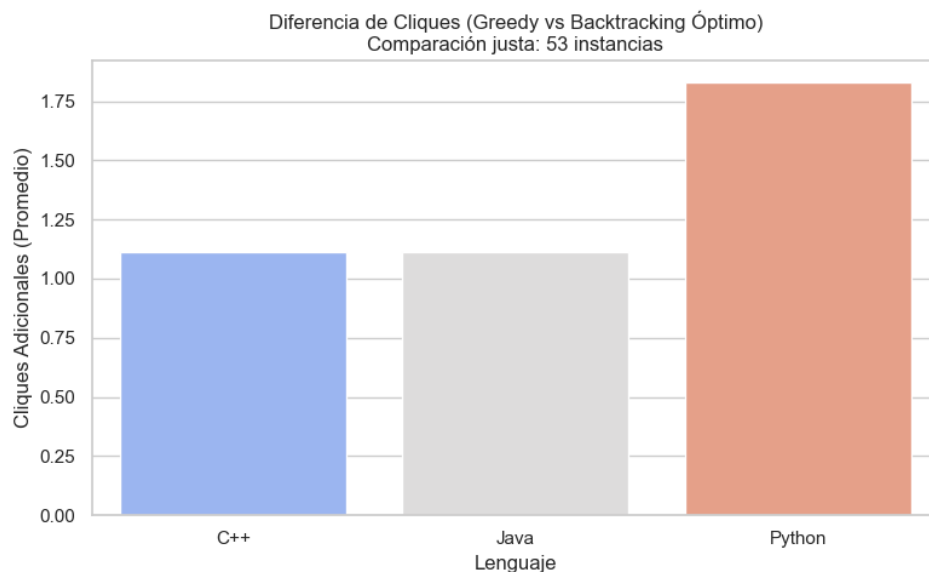
6.3. Comparación de Enfoques

Para evaluar la calidad de las soluciones generadas por la heurísticas Greedy, se comparó el número de cliques obtenidos mediante dicha solución mejorada y el número de cliques correspondiente al algoritmo backtracking. Es importante recalcar que el objetivo que se busca con *Clique Cover* es minimizar el número de cliques, por lo que una menor diferencia espacial entre Greedy respecto a backtracking indica una heurísticas más precisa.

6.3.1. Análisis de Calidad de la Solución (Diferencia de Cliques)

El siguiente gráfico ilustra la cantidad promedio de cliques adicionales que la solución Greedy genera en comparación con el óptimo teórico, tras evaluar 53 instancias.

Figura 4: Diferencia promedio de cliques generados (Greedy vs. Backtracking Óptimo) para 53 instancias.



Fuente: Elaboración propia basada en los resultados obtenidos tras ejecutar tools benchmark.

Como es posible observar en la Figura 4, se nota la existencia de una penalización en la calidad de la solución cuando se utiliza el enfoque Greedy, la cual va variando según el lenguaje de programación y sus respectivas implementaciones de la heurística:

- **C++ (~1.1 cliques adicionales):** Procesa los nodos de forma secuencial (0 a $n - 1$). Obtiene una cota de error baja, cercana a la solución óptima.
- **Java (~1.1 cliques adicionales):** Presenta un comportamiento prácticamente idéntico a C++, debido a una lógica secuencial similar en el procesamiento de los vértices.
- **Python (~1.8 cliques adicionales):** Muestra el mayor error, a pesar de utilizar ordenamiento previo por grado, lo que no logra mejorar la calidad de la solución en estas instancias.

6.3.2. Discusión del Compromiso Tiempo vs. Calidad

Los resultados obtenidos logran evidenciar el clásico *trade-off* en la resolución de problemas *NP-completos*:

1. **Fallo de la heurística de grado (Python):** El ordenamiento por grado descendente no resulta efectivo en estas instancias, generando más cliques al fragmentar el grafo tempranamente.
2. **Eficiencia computacional:** Aunque no se alcanza el óptimo, el tiempo se reduce drásticamente respecto a backtracking. Para escenarios donde se tolera un pequeño error, el enfoque greedy secuencial ofrece un buen balance entre calidad y rapidez.

7. Conclusión

Se pudo apreciar que el problema de Clique Cover por su naturaleza de problema NP-completo, los tiempos para hallar una solución óptima escalan de forma exponencial si se usa soluciones que buscan de forma exhaustiva los cliques óptimos, a diferencia de la solución mejorada, esta al tener un enfoque avaro, puede buscar soluciones que pueden llegar a ser óptimas o muy cercanas a esta en un tiempo infinitamente menor a la solución base.

Con respecto a los lenguajes en los que se implementaron las soluciones, la forma en la que esta construido y como funciona afecta también el como opera las soluciones, se pudo ver con el diseño del algoritmo implementado en el lenguaje de programación C++ que su resultado fue el peor para usar la solución base debido a su enfoque de mayor recorrido lo que llevo a tiempo de ejecución mas alto. Pero a su vez ser el mejor para usar la solución mejorada, a diferencia de Java que es el mejor en backtracking y Python que es mas balanceado tomando en cuenta las grandes diferencias que poseen los lenguajes de programación entre si. Dejando en evidencia la utilidad que podría tener cada uno de estos en determinados problemas, uso y enfoque con el que se quiere buscar una solución a un problema tomando en cuenta la escalabilidad de estos problemas de carácter complejo.

Concluyendo, para problemas de esta naturaleza, en el que el tiempo de para encontrar una solución escala de forma exponencial a medida que el problema se hace más grande, si se quiere buscar una solución en poco tiempo y que sea cercano a lo optimo, se puede usar C++ con enfoque avaro, mientras que si se quiere la mejor solución posible aunque se tenga que esperar un poco de tiempo, es mejor usar Java con enfoque exhaustivo, dejando a Python como la peor opción a considerar ya que no se especializa ni en buscar soluciones rápidamente ni en buscar la mejor de todas.

Referencias

- [1] Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. San Francisco, CA: W. H. Freeman.
- [2] Diestel, R. (2017). *Graph theory* (5th ed.). Springer.
- [3] West, D. B. (2001). *Introduction to graph theory* (2nd ed.). Upper Saddle River, NJ: Prentice Hall.
- [4] Wolsey, L. A. (1998). *Integer programming*. New York, NY: Wiley-Interscience.
- [5] Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller & J. W. Thatcher (Eds.), *Complexity of computer computations* (pp. 85–103). New York, NY: Plenum Press.